The Multitron and Megatron

Nandan Goyal

October 2025

1 Multitron

A Multitron is a stack of Neurons, and can be represented by a matrix of weights where each row contains the weights for the corresponding neuron, with the first column containing biases. It represents a linear function from $\mathbb{R}^n$ to $\mathbb{R}^m$.

$$\vec{M}(\vec{x}) = [W]\vec{n}$$
$$\vec{n} = \begin{bmatrix} 1 \\ \vec{x} \end{bmatrix}$$

2 Multitron Training

A Multitron can linearly approximate any data mapping two vector spaces. We start with random weights. Define the loss for a given data point $(\vec{x}, \vec{y})$ as:

$$L = \tfrac{1}{2}(\vec{M}(\vec{x}) - \vec{y})^2$$

We calculate the gradient of L with respect to $[W]$, and then move $[W]$ in the direction of the negative gradient to move towards an error minima. By the chain rule,

$$\frac{\partial L}{\partial W_{ij}} = \sum_{\forall i'} \frac{\partial L}{\partial M_{i'}} \frac{\partial M_{i'}}{\partial W_{ij}}$$

Here, W_{ij} is the weight in the i-th row and j-th column, $M_{i'}$ is the i'-th element of $\vec{M}(\vec{x})$. Notice that the second partial derivative is zero if $i' \neq i$. Hence,

$$\frac{\partial L}{\partial W_{ij}} = \frac{\partial L}{\partial M_i} \frac{\partial M_i}{\partial W_{ij}}$$
$$\frac{\partial L}{\partial M_i} = M_i - y_i$$
$$\frac{\partial M_i}{\partial W_{ij}} = n_j$$

In matrix form,

$$\nabla_{\vec{M}} L = \vec{M} - \vec{y}$$
$$\nabla_{[W]} L = (\nabla_{\vec{M}} L)(\vec{n}^{\mathsf{T}})$$

3 Megatron

A Megatron (neural network) is a series of Multitrons that each take the output of the previous Multitron as their input, except that each Multitron runs its output through an activation function, which gives the final "activated" output, which is what actually gets fed into the next Multitron. We have,

$$\vec{O}^{(m)}(\vec{x}) = \vec{A}^{(m)}(\vec{Z}^{(m)})$$
$$\vec{Z}^{(m)}(\vec{x}) = [W^{(m)}]\vec{N}^{(m-1)}(\vec{x})$$
$$\vec{N}^{(m-1)}(\vec{x}) = \begin{bmatrix} 1 \\ \vec{O}^{(m-1)}(\vec{x}) \end{bmatrix}$$

where $\vec{O}^{(m)}$, $\vec{A}^{(m)}$, $\vec{Z}^{(m)}$ and $[W^{(m)}]$ are the (activated) output, activation function, unactivated output and weight matrix of the m-th Multitron, respectively. Clearly, the output of the m-th Multitron depends on the output of the $(m-1)$-th Multitron. For a Megatron of n Multitrons, the final output is simply $\vec{O}^{(n)}(\vec{x})$.

4 Megatron Training

Once again, we define the loss as:

$$L = \tfrac{1}{2}(\vec{O}^{(n)}(\vec{x}) - \vec{y})^2$$

For the m-th Multitron, we know that,

$$\frac{\partial L}{\partial W_{ij}^{(m)}} = \frac{\partial L}{\partial O_i^{(m)}} \frac{\partial O_i^{(m)}}{\partial W_{ij}^{(m)}}$$
$$\frac{\partial O_i^{(m)}}{\partial W_{ij}^{(m)}} = A_i'^{(m)} N_j^{(m-1)}$$

where A_i' is the derivative of the activation function function applied to the corresponding unactivated output.

We cannot evaluate $\partial L/\partial W_{ij}^{(m)}$ directly since L depends on $[W^{(m)}]$ indirectly via the composition chain. We must invoke the chain rule:

$$\frac{\partial L}{\partial O_i^{(m)}} = \sum_{\forall i'} \frac{\partial L}{\partial O_{i'}^{(m+1)}} \frac{\partial O_{i'}^{(m+1)}}{\partial O_i^{(m)}}$$
$$\frac{\partial O_{i'}^{(m+1)}}{\partial O_i^{(m)}} = A_{i'}'^{(m+1)} W_{i'i}^{(m+1)}$$

This is exactly what we need since this allows us to calculate $\partial L/\partial O_i$ for one Multitron in terms of all the $\partial L/\partial O_i$ for the next Multitron. Since computing $\partial L/\partial O_i^{(n)}$ is trivial, we can successively compute this gradient for all the Multitrons, going backward one at a time. Once we have computed all $\partial L/\partial O_i$, we can compute all the $\partial L/\partial W_{ij}$, which is what is needed for learning.

If we define $\vec{\Delta}^{(n)} = \nabla_{\vec{O}^{(n)}} L$, then in matrix form,

$$\vec{\Delta}^{(n)} = \vec{O}^{(n)} - \vec{y}$$
$$\vec{\Delta}^{(m)} = \vec{R}(\mathrm{diag}(\vec{\Delta}^{(m+1)} \odot \vec{A}'^{(m+1)})[W^{(m+1)}]_{-c_0}), m < n$$
$$\nabla_{[W^{(m)}]} L = (\vec{\Delta}^{(m)} \odot \vec{A}'^{(m)})(\vec{N}^{(m-1)})^{\mathsf{T}}$$

where $\odot$ is the Hadamard product, the subscript $-c_0$ means removing the first column of the matrix (done to skip the biases), and the function $\vec{R}$ returns the vector formed by summing all the elements of a matrix along its rows.

Note that the second equation can be rewritten as:

$$\vec{\Delta}^{(m)} = [J_{\vec{O}^{(m)}}](\vec{O}^{(m+1)})^{\mathsf{T}}\vec{\Delta}^{(m+1)}$$

where J is the Jacobian operator. This isn't a special formula for Megatron training, but rather just the generalized form of the chain rule applied in this context. If you expand the Jacobian and rewrite terms, you get the closed form as written above.